

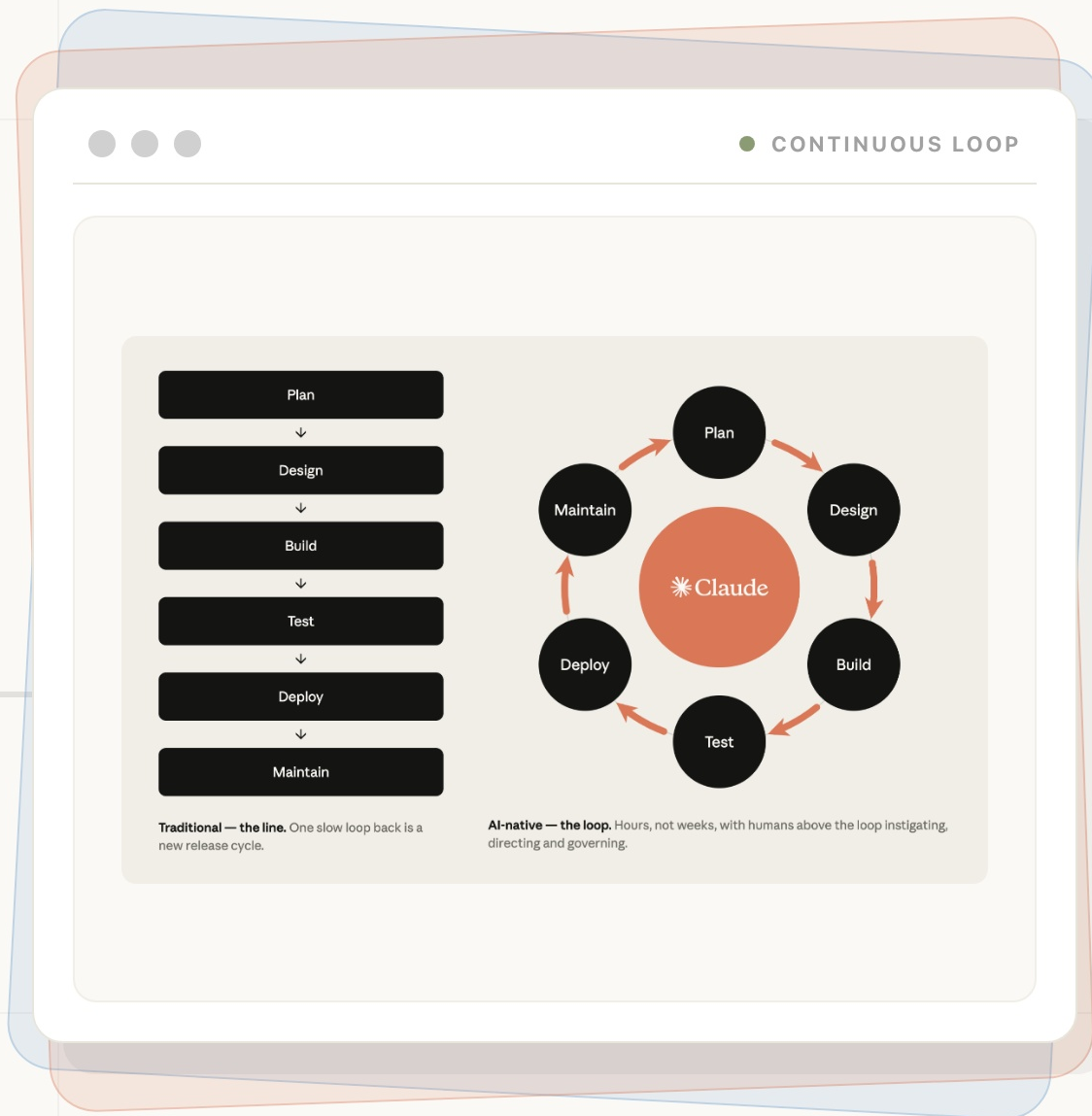
AI 原生 软件开发生命周期 SDLC.

代码快了之后，如何重构

需求、审查、发布与运维闭环



分享
碳硅同传



01

代码不再是瓶颈，
交付系统才是



代码生成变快后，瓶颈为什么向两端迁移？

瓶颈的左移与右移

⚡ 高速区间

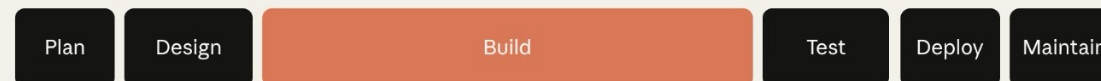
Build 环节已被 AI 工具极致压缩至小时级。

⌚ 人速瓶颈

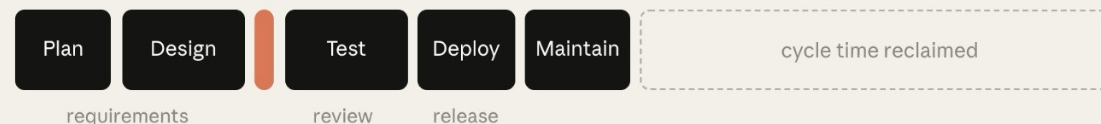
Plan (需求) / **Test** (测试) / **Deploy** (发布) 仍受限于人力流转速度。

⚠ 交付断层：代码的高产出，被两端的人工等待时间完全抵消。

Before agents — every stage runs at human speed



After agents — build runs at agent speed



🔍 点击放大查阅图表详情

传统 SDLC 的控制逻辑，为什么开始跟不上 Agent 的产出速度

旧假设与新现实的冲突

	传统 SDLC 模式		⚡ AI 原生 / Agent 时代
核心假设	编写代码最耗时、成本最高	→	生成代码速度极快、成本低
审查方式	逐行人工审查	→	代码产量远超人力逐行审查极限
治理结果	周会/月会处理例外审批	→	审批队列严重堆积，或代码带险上线

8 倍代码量、约 80% Claude 代码：Anthropic 已经面对什么规模变化

Anthropic 内部研发实证数据

8 倍

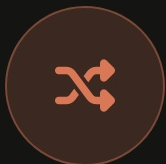
—
工程师人均季度
交付代码量增幅

约 80%

—
目前合并代码中
由 Claude 编写的占比

>50%

—
由内部版 Claude Tag
直接发起的合并动作



人的角色转移

工程师精力全面转向 **指挥、设定意图与最终批准**

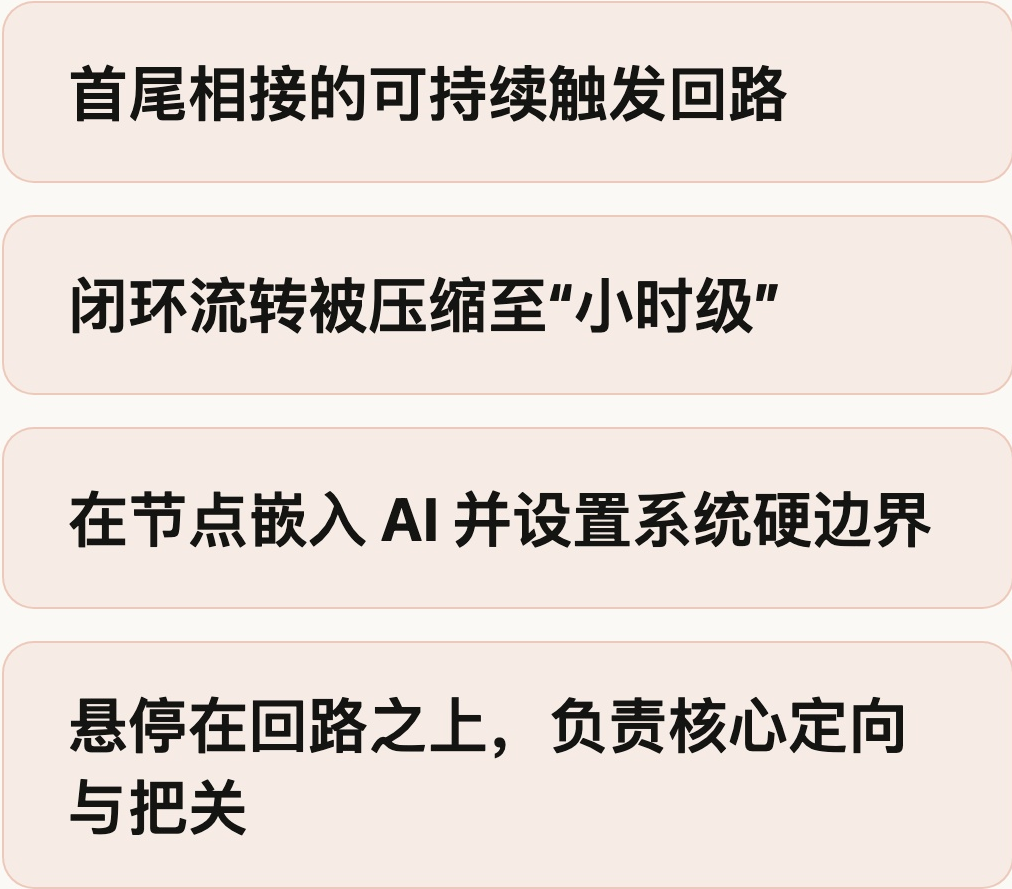
AI 原生 SDLC 的运转逻辑

从线性流程到持续闭环

传统瀑布流 →



🔄 AI 原生闭环



02

真正连接六个阶段的， 是版本化产物链

在超高速运转的回路里，摒弃口头交接与无休止的对齐。



一条可追溯的交付链

代码库里的文档接力赛



机器与人双向可读

全系采用 Markdown 作为前置阶段产物，不仅产品负责人能轻松看懂，Agent 也能零门槛读取完整的开发上下文。



原生自带审计闭环

每一个构建阶段都由文件写入结束与读取开始，形成了一本清清楚楚、随时能翻查账本的强悍交接追溯链。

动作与触发机制

文件状态即“发令枪”

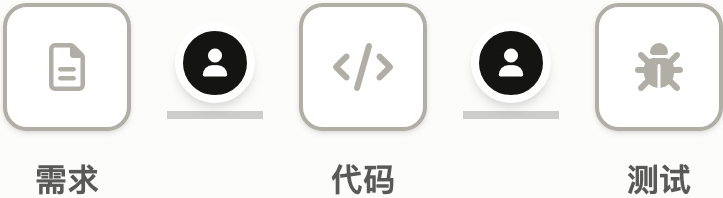
接受 ACCEPT	intent.md 被接受	→ 自动触发需求与设计
批准 APPROVE	spec.md 被批准	→ 启动计划 (Plan) 模式
合并 MERGE	PR 成功被合并	→ 启动自动化流水线
异常 BREACH	生产指标 越出控制带	→ 自动生成 intent.md

人的角色没有消失：判断权随着审查产物向后移动

人机协作边界重构

过去 / 单向线性

人工主导推进流程 逐行审查代码执行细节



人类深陷执行链路，成为流转瓶颈

现在 / 自动闭环

Agent 推进节点流转 人类悬停在回路之上把关



Agent 高速自动回环



注意力不再纠缠于 执行细节，而是转移到对意图与风险的 核心产物审查。

系统在跑，脏活累活交给 Agent，但拍板决定的裁判权始终在你手里。

遗留系统的三种接入策略

永远只要一个唯一真源 (Source of Truth)



1

代码库为主

真源归属

Git Repo

协同方式

遗留系统仅引用 Markdown 的 Commit SHA。



2

老系统为主

真源归属

Jira / ServiceNow

协同方式

老系统主导，Agent 通过 MCP 接口同步结果回 Markdown。



3

底线方案

真源归属

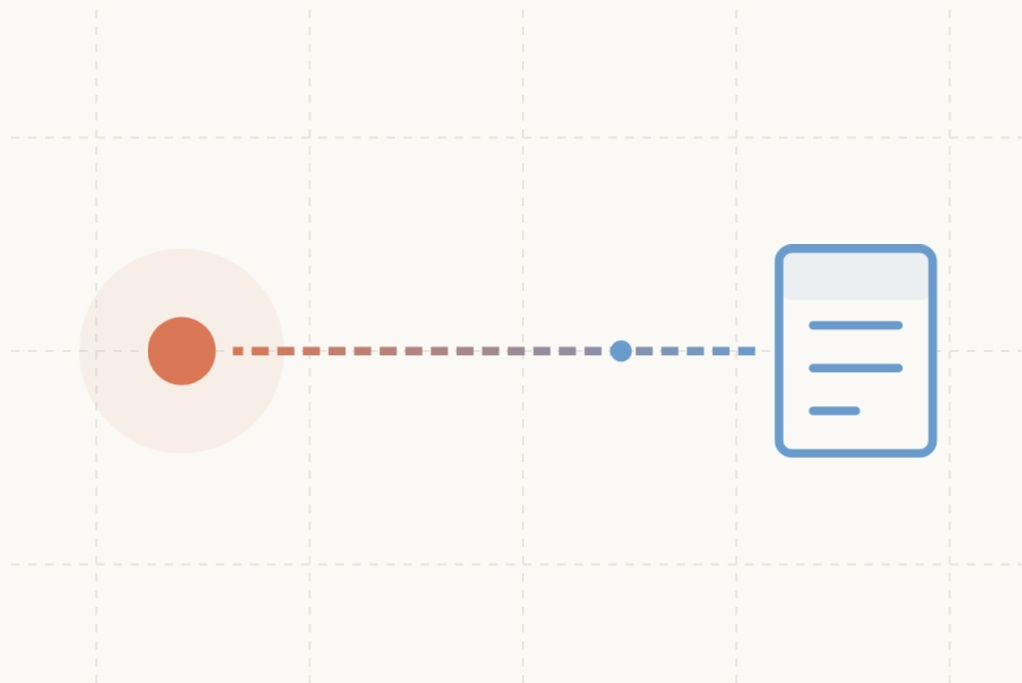
双轨并行 (过渡期)

协同方式

互相链接：Artifact 记录工单 ID，老系统记录 SHA。

03

Plan + Design:
先把意图变成可以
交接的决定



不要让想法排队等 PRD： 先把提出者原话沉淀成 `intent.md`

需求无需等待，想法即刻落盘

01 核心原则

捕捉提出者原话，跳过漫长的文档排期等待。

02 呈现载体

使用 `intent.md`
产品负责人与 Agent 均可双向无障碍阅读。

03 存放策略

推荐在代码库设立专属 `intent/` 目录，让需求紧贴未来衍生出的代码。

意图的准入与判断

AI 负责起草，人类负责拍板

⚠ 流程红线

无论意图来自人力还是事件触发，提交前**必须经由产品负责人审阅修正**。

核心滞后指标

意图存活率

衡量被产品负责人最终接纳，并进入下游设计的
的 `intent.md` 比例。

角色定位演进

过去 写需求文档的人



核心 做方向判断的人

需求与设计：从分离到合并

阶段塌缩，前置合规

传统模式

需求与设计分离
多团队交接耗时

数周后的评审会上
才发现政策问题

传递节点多
信息损耗严重



阶段关系



合规切入点



信息损耗

AI 原生模式

需求与设计
合并至单次会话

在 `spec.md` 生成时
即刻应用

意图直达设计
实现无缝衔接

高风险前置解决，审计要素打包入库

审计透明，风险闭环

workflow约束

优先解决 **Agent** 标注的高风险点，扫清障碍后再交接工程团队。

最终决定权

由人类队友（产品或技术负责人）签核，确认无风险后放行进入构建。

硬核审计链：三位一体的版本控制

有账可查，全链路透明



spec.md

最终输出的产物，研发构建的直接规格依据。



Prompt

生成该文档所使用的上下文，保留原始意图与动机。

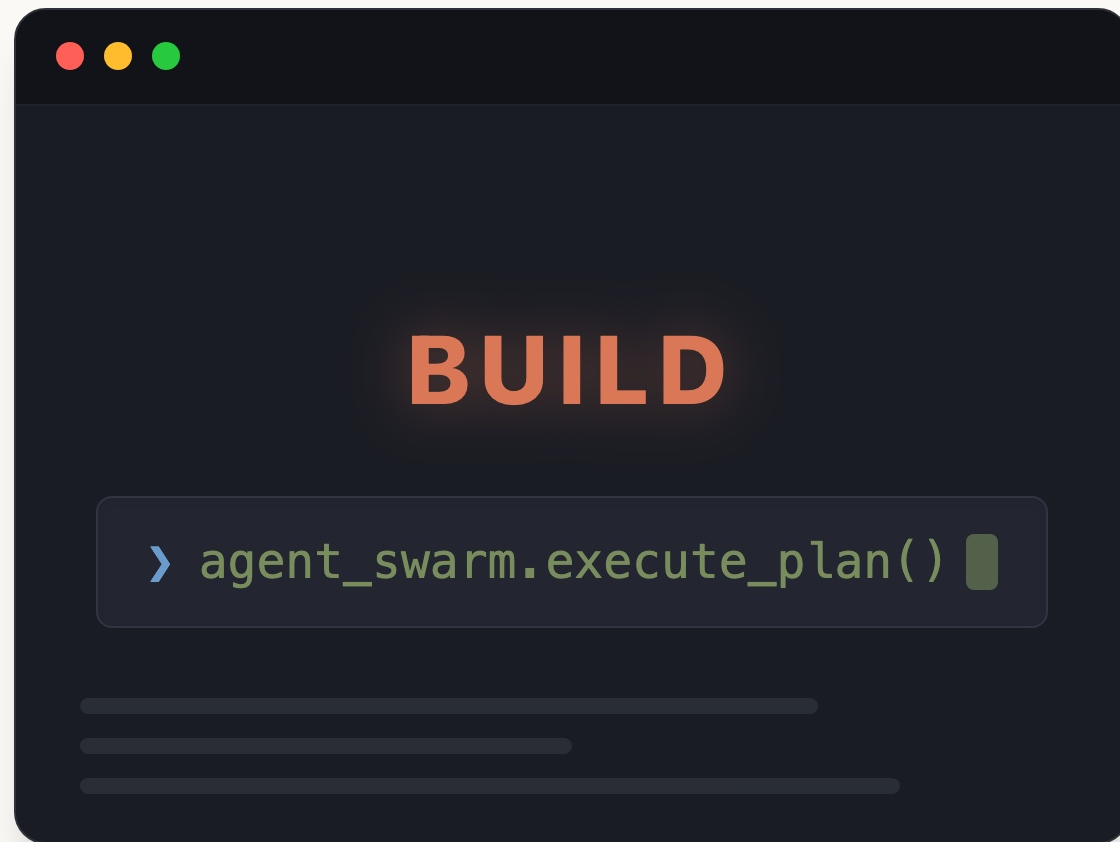


Skill 版本

当时生效的组织政策约束文件，界定边界与合规要求。

04

Build:
先审计划，再让
agent 大规模写代码



没有被接受的 plan.md 就不实现：把第一次正式审查放到代码生成之前

从“写完审代码”到“动手前审计划”



“没有被接受的
plan.md，
一行代码都不准写”

这不仅是纪律，更是一道前置的流
程控制阀。

01

流程前置

第一次正式审查强制发生在 代码生成之前，取代传统的“事后复盘”。

02

Plan Mode 约束

Agent 仅拥有 只读权限，完全被剥夺修改代码库的物理能力。

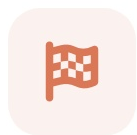
03

硬性底线

无 `plan.md` 批准授权，绝不执行任何实现动

一份合格计划的标准

消除上下文依赖的“硬核”验收标准



验收基准

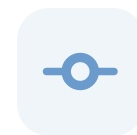
脱离历史对话记录，
第三方开发者仍具备
完全可执行性。



执行预期

计划足够扎实，代码
实现往往**一次性通**
过。

Single Pass



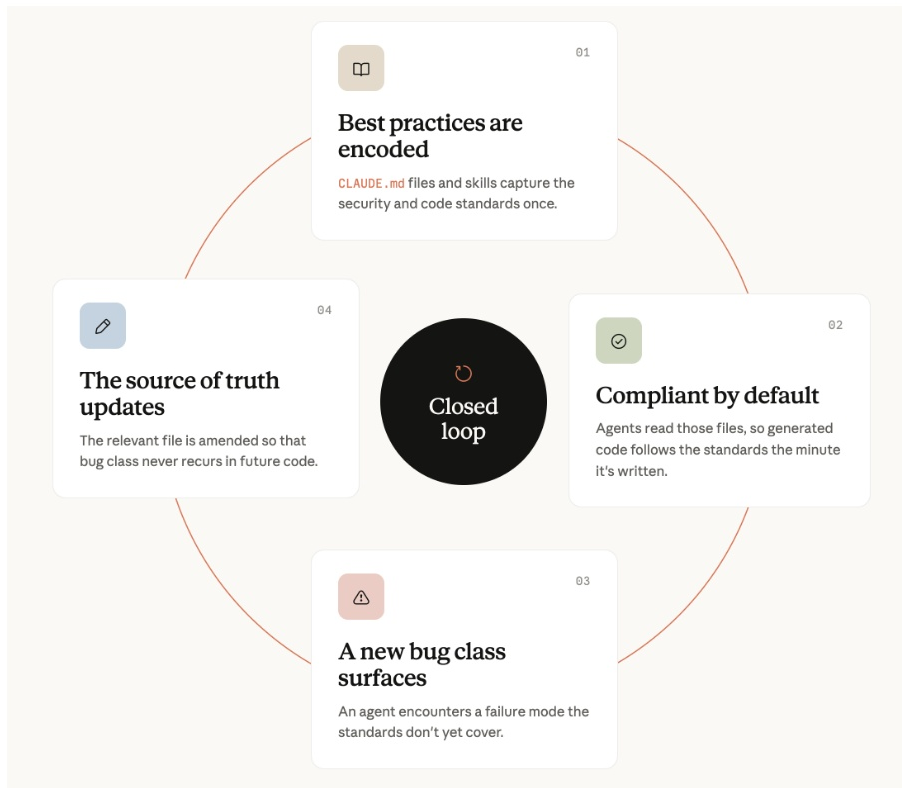
同步机制

若实现偏离计划，必
须在**同一次 Commit**
中更新 `plan.md`。

🛡️ 可通过 hook 强制校验

CLAUDE.md、Skill 与 Prompt 各自应承载什么知识？

机构知识的三层剥离与存放



i Anthropic 官方知识闭环：最佳实践通过版本化文件形成合规基础，从而让 Agent 生成的代码默认安全。

Prompt 第一层

01

应对单次特定任务的上下文

 随手编写，即用即弃

CLAUDE.md 第二层

02

团队级约定、架构指导与常见避坑指南

 团队共建，严格控制在一页以内

Skill 第三层

03

组织级强制合规要求 (如核心安全政策)

 集中管理，严格接入版本控制

Skill 是建议性控制，Hook 才是必须持续成立的确定性边界

双重保险：从“不要做”到“做不到”

💡 大模型理解层

Skill

建议性控制

ADVISORY CONTROL

引导行为，使违规事件变得
罕见。

+

🔒 代码执行层

Hook

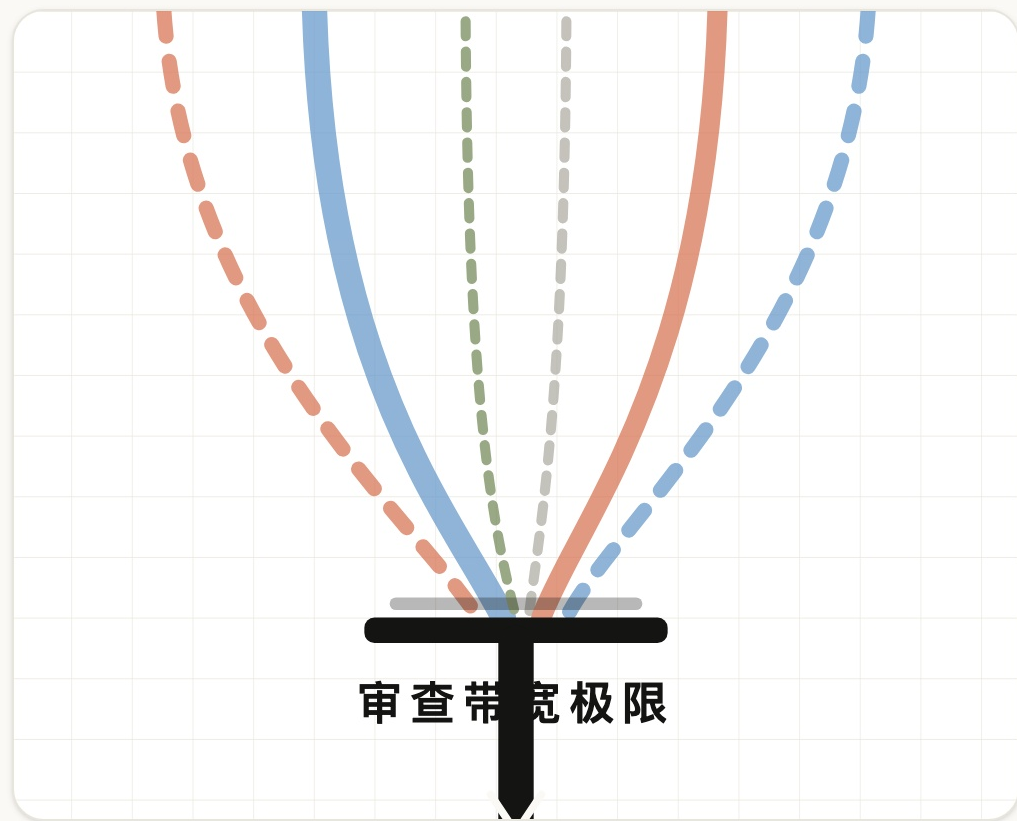
确定性边界

DETERMINISTIC BOUNDARY

物理拦截，使违规行为变成
不可能。

并行代码可以狂奔 但交付通道只有一条

算力无上限，但人类注意力是稀缺资源



01

谨慎起步，切勿盲目并发

不要试图单纯用 Session 数量换取产出。建议初开只跑 2-3 个并发会话试水，避免生成泛滥将仓库出口彻底堵死。

02

认清新的天花板：审查带宽

产能的瓶颈已经转移。它不再取决于 Agent 写代码有多快，而是受限于人类能够认真审阅和验证的极限带宽。

03

管控模式必须升级至全局配置

随着并行量逐步上升，人工“盯盘”模式注定失效。控制逻辑必须向上收敛，沉淀为代码仓库级别的全局规则与自动化拦截。

05

Test + Review:
生成速度上来了，
验证必须更早到达



让 Session 在交付前形成反馈回路

反馈必须前置



传统模式

滞后的流水线反馈

🕒 反馈速度

CI 延迟几分钟，测试人员延迟数天

👥 审查压力

集中在人类身上，极易造成瓶颈堵塞

🔑 验证时机

离开会话，进入代码合并流水线之后



AI 原生模式

Session 内的即时回路

🕒 反馈速度

动作执行后，**立刻**通过测试/构建完成自查

👥 审查压力

Agent **自动自我修正**，降低人类审查噪音

🔑 验证时机

在当前 Session 内，**交付给人类看之前**

验证机制的双轨策略：代码强控与视觉反馈



Bug 修复回路

• 强控制规则流



复现 | 生成必然失败的测试用例



固定 | 先提交测试，保留失败证据



修复 | 修改实现代码以通过测试



阻断 (Hook 防御)

强制禁止 Agent 在修复期间编辑测试文件，严密保护检验条件不被削弱。



UI 开发回路

• 循环视觉反馈流



实现 编写 UI 代码



截图 渲染实际画面



机器对比 比对设计原稿



调整 微调样式细节



通常预期迭代 **2-3** 轮达到最终可用状态

建立 Verifier 独立防线

破除上下文盲区

主 Agent

上下文状态

拥有完整推理历史

包含对话、试错与推理过程。

验证风险

受既往假设与自身逻辑影响，自己查自己极易陷入 **逻辑盲区**。



Verifier Subagent

上下文状态

完全清空的新窗口

不携带任何先入为主的生成过程记忆。

核心优势

只看最终产物，不受过程假设干扰，确保结论 **绝对客观独立**。

把 CLAUDE.md、skills 和 hooks 也放进 eval：配置同样会回归

配置也是代码，需要回归测试



被测控制对象

CLAUDE.md

团队约定与上下文规约

Skills

需要一致应用的组织级政策与技能

Hooks

必须持续成立的确定性动作边界



Eval 运行机制

AI 原生 QA



测试底座

提取 20-50 个 近期真实任务及其预期结果



触发条件

按计划定时执行，或在配置发生任何变更时触发



安全门禁

强制拦截

导致通过率下降的 Skill 修改，必须被拦截审阅

AI 时代的 PR 审查：职责边界不可逾越

效率的增益，并未自动转化为更好的评审质量

实证数据警示

102 万+

PR 演变研究样本

Agent 参与协作显著加快了整体评审的**决策速度**，但核心结论是：人类的把关不可或缺，审查的判断维度必须**向上层转移**。



AI 角色：全量执行者

 执行所有 PR 的**全量策略检查**，覆盖基础语法与规范。

 发现问题后，按**严重度排序**输出，提供结构化审查建议。



人类专家：最终把关人

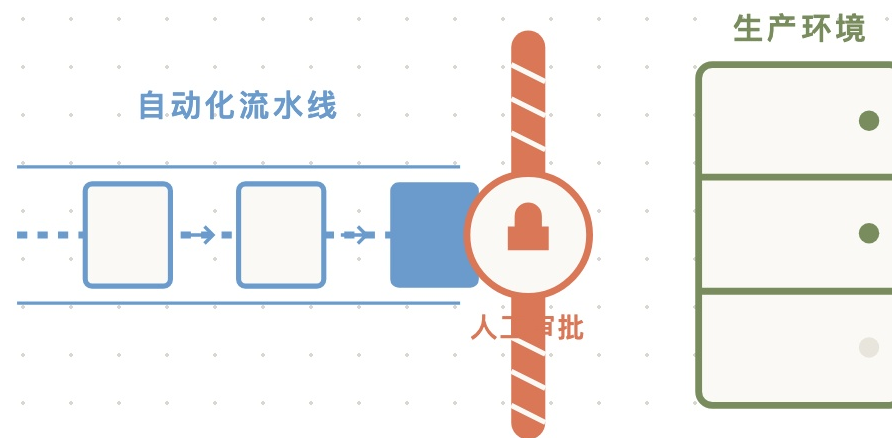
 通过分支保护，牢牢掌握系统的**最终批准权**。

 不再逐行看语法，专注判断代码是否偏离**计划意图与风险可控性**。

 **写代码的 Agent，绝不能有权限批准自己的 PR**

06

Deploy:
自动化可以走到生
产门口，但不能自
己跨过去



Approval Gate 应该放在哪里？

Build 阶段阻断，Deploy 阶段才让人做发布判断

01 BUILD PHASE 构建阶段 (Build)

Hook 行为策略

允许 / 阻断 (无人工干预)

系统底层设计目的

保持并行会话高速推进，绝对不让人成为整条自动化流水线的阻塞点。

静默运行



02 DEPLOY PHASE 部署阶段 (Deploy)

Hook 行为策略

暂停并询问 (需人工批准)

系统底层设计目的

强制人类介入关键节点，完成最终的发布判断与安全守门。

安全准入



Agent 权限防线：为什么不能只靠 Prompt?

深度防御体系



Prompt 指令引导 易被攻破

仅能做意图引导，**防不住恶意注入 (Prompt Injection)**。一旦防线被绕过，Agent 可能执行危险操作。



应用层：工具权限配置 动作拦截

配置 Allow / Deny 列表，限制 Agent 的常规动作边界。有效防止越权调用敏感工具，降低弹窗带来的疲劳轰炸。



系统层：沙箱与网络边界 物理隔绝底线

OS 级沙箱环境与域名白名单控制。真正物理隔绝未授权的网络出口，是**堵住凭据泄露的最终底线**。

开发、预发布、生产采用不同 Autonomy Tier: Agent 的动作边界随环境变化

Agent 自主权的分层策略

开发环境

Agent 权限

自由部署
高自主权

允许 Agent 自由折腾，动作不受限。

- 人类干预要求
无需干预

预发布环境

Agent 权限

受限部署
强制回滚演练

动作开始受限，必须在此环境中反复演练一键回滚机制。

- 人类干预要求
需复核与回归验证

生产环境

Agent 权限 (极度克制)

仅限“准备就绪”
绝对禁止跨界

自动化可以走到生产门口，但规矩是死的：绝对不能自己跨过去。

- 人类干预要求 (硬性死线)
必须具名经理手动授权

Auto Mode 关键测试数据

人类审查的脆弱性

97%

日常权限提示通过率

面临海量提示极易产生“习惯性盲签”

13.6%

危险命令人工拦截率

在超千人受控盲测中防线几近崩溃

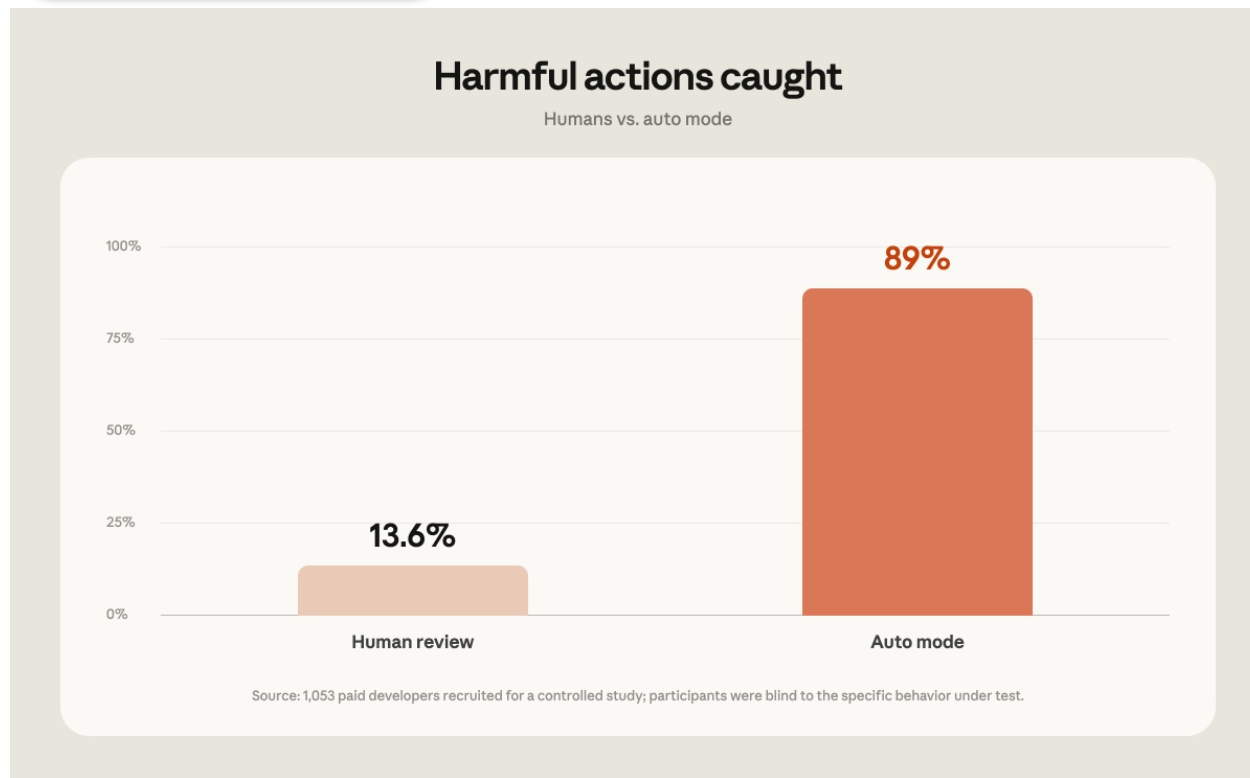
⚠ 核心发现

17% ↘ 5%

拦截率随会话时间断崖式衰减

随着上下文拉长，人的注意力愈加脆弱

📊 官方受控盲测图表



🔍 点击图片放大查看实验细节

自动模式的防御与局限

代码审查危险命令拦截率分析

● 人工审查

拦截率

13.6%

漏检风险

随精力衰减，极易出现疲劳盲签。

● Auto Mode 自动审查

拦截率

89%

核心缺陷

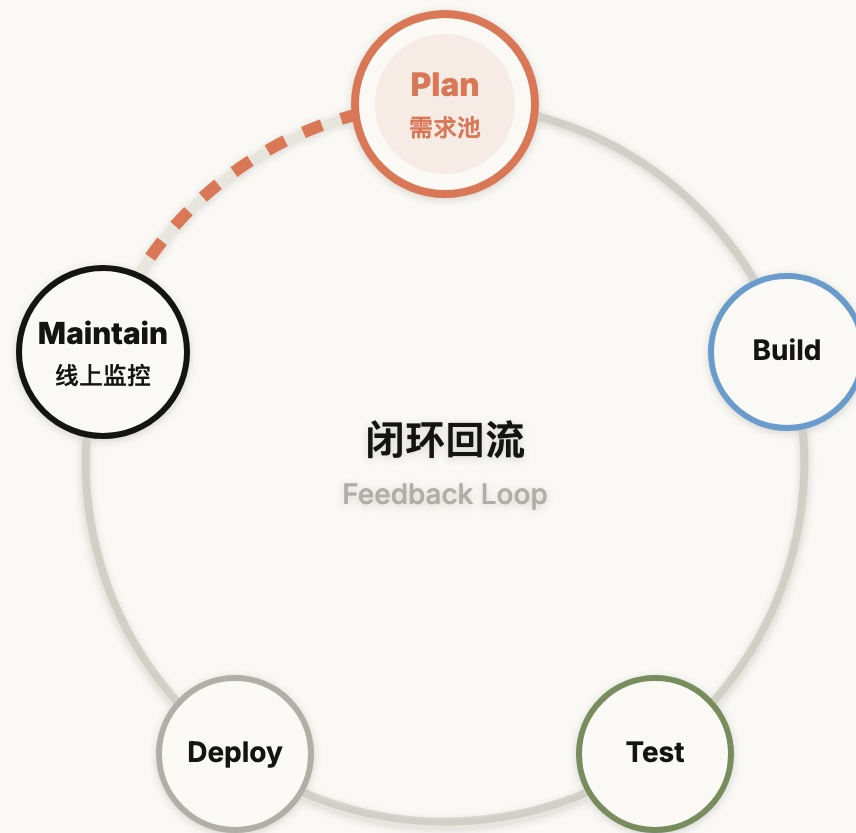
在对抗性攻击集里，依然存在 7% 的漏检率。

底线原则

自动化仅可前置过滤低级错误，
高风险生产变更 必须保留人工最终审查。

07

Maintain:
闭环真正成立之后
新的问题才开始



控制带越界后的自动诊断

按指标偏离程度分层响应



i 机制说明：全过程监控与日志分析均为 非交互运行，无需人工唤醒。

线上反馈统一收口

所有线上反馈最终回归同一入口

1

触发源头汇聚

定期扫描

异常告警

协同排查

2

沉淀统一格式



intent.md

3

业务决策与闭环

负责人像评审需求一样决策，一旦确认修复，立即滚回并形成新一轮迭代的起点。

#inc-checkout incident channel · Claude is a member under its own identity

RM

R. Mehta 22:04

Checkout 5xx rate has been climbing since the 21:40 deploy. @claude take a look?



Claude 22:07

On it — comparing error signatures against the deploy diff. Cause looks like the new cache key dropping the tenant id. The rollback was rehearsed in staging this morning; shall I run it?

RM

R. Mehta 22:08

Go.



Claude 22:12

Rolled back. The 5xx rate is back inside its band, and the post-mortem is written to lessons/2026-06-checkout-cache.md.

规格不是越多越好

避免解释漂移与上下文退化

⚠️ 散文规格的隐患

解释漂移

上游 Agent 误解 10%，下游完全走偏

上下文退化 (Context Rot)

冗长历史废案导致模型指令混淆

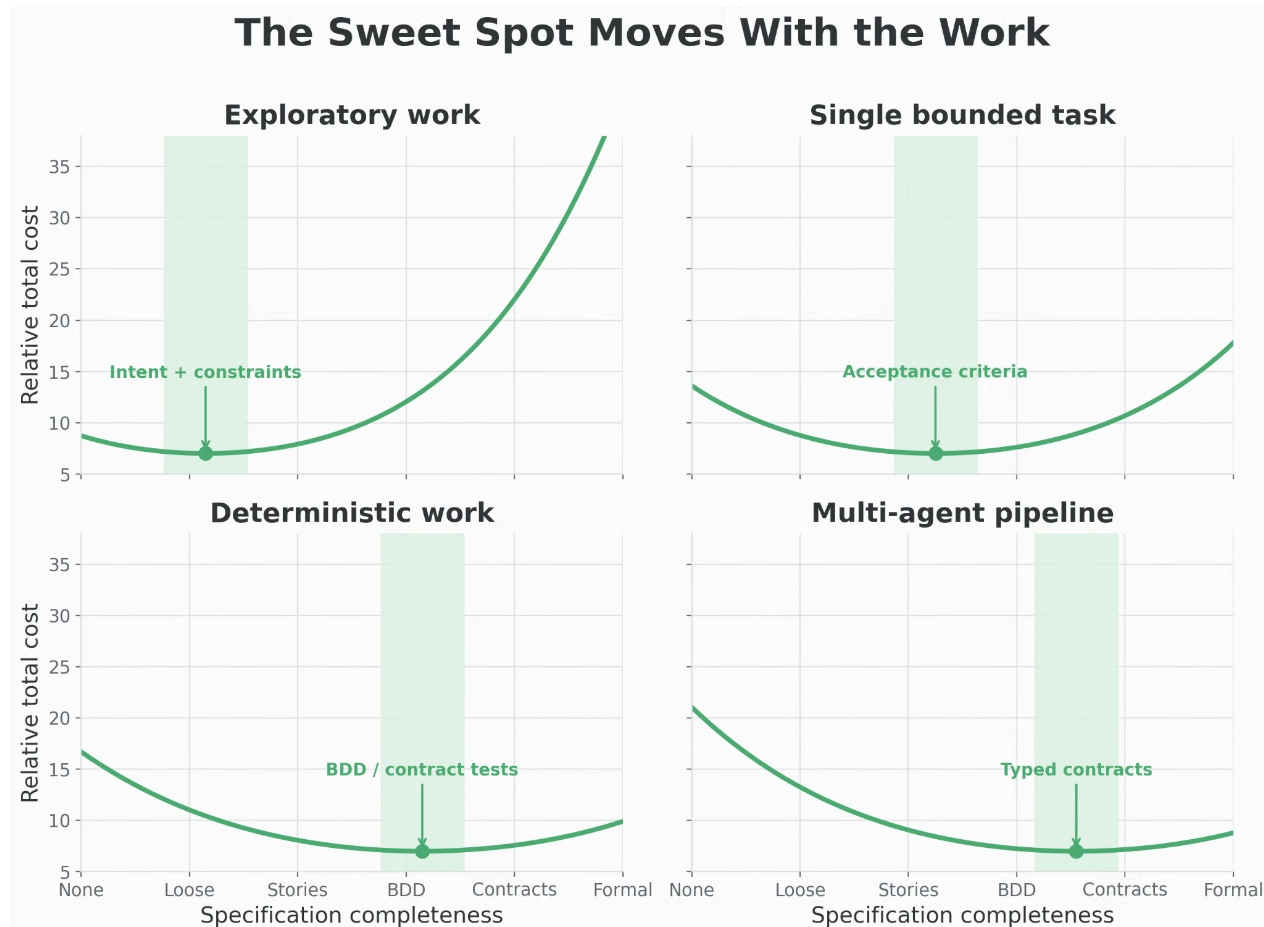
🎯 按任务配置规格

探索性工作

给意图 + 设定边界

多 Agent 流水线

强类型契约 + 可执行验收



不同工作类型对应的最佳规格颗粒度

全链路度量法则：代码产出狂飙 ≠ 组织价值提升

警惕局部指标陷阱

01 虚假繁荣

PR 评审时长

骤降 ↓

代码批准率

猛增 ↑

⚡ 导致：习惯性放水

数月后



02 延迟代价



变更失败率显著反弹

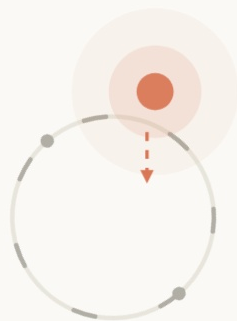
Change Failure Rate ↗

核心洞察

系统视角

缺乏全链路度量的自主性，只是更快的熵增。

必须将「代码产出效率」与「发布质量」进行对冲度量。



人的判断，始终在闭环之上

代码生成可以无限快，但对意图和风险的把控，
永远是不可被替代的核心环节。



碳硅同传



赞同



收藏



探讨